

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №24

Тема: Створення алгоритмів знаходження сум і добутків

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати алгоритми обчислення скінченних і нескінченних сум та добутків: накопичувачі, рекурентне обчислення членів ряду, обчислення суми з заданою точністю ε ; навчитися перевіряти результати за формулами та досліджувати вплив порядку підсумовування на точність.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Накопичення суми й добутку. Сума $S = a_1 + a_2 + \dots + a_n$ обчислюється циклом: спочатку $s = 0$, потім на кожній ітерації $s += a_k$. Добуток $P = a_1 \cdot a_2 \cdot \dots \cdot a_n$: спочатку $p = 1$, далі $p *= a_k$. Початкові значення — нейтральні елементи відповідних операцій (0 для додавання, 1 для множення); для порожньої суми результат — 0, для порожнього добутку — 1.

2. Рекурентне обчислення членів. Якщо кожен член ряду виражається через попередній, то обчислювати його «з нуля» (ступінь, факторіал) неефективно. Наприклад, для ряду $e^x = 1 + x/1! + x^2/2! + x^3/3! + \dots$ маємо $a_k = a_{k-1} \cdot x / k$, $a_0 = 1$. Обчислення за рекурентною формулою потребує $O(1)$ операцій на член (замість $O(k)$ при обчисленні степеня і факторіала), а також запобігає переповненню при обчисленні великих факторіалів.

Функція	Ряд Тейлора	Перший член a_0	Рекурентне співвідношення
e^x	$\sum x^k / k!$	1	$a_k = a_{k-1} \cdot x / k$
$\sin x$	$\sum (-1)^k x^{2k+1} / (2k+1)!$	x	$a_k = -a_{k-1} \cdot x^2 / ((2k)(2k+1))$
$\cos x$	$\sum (-1)^k x^{2k} / (2k)!$	1	$a_k = -a_{k-1} \cdot x^2 / ((2k-1)(2k))$
$\ln(1+x), x <1$	$\sum (-1)^{k+1} x^k / k$	x	$a_k = -a_{k-1} \cdot x \cdot (k-1) / k$

3. Обчислення суми з точністю ε . Для збіжного ряду підсумовування припиняють, коли черговий член за модулем стає менший за ε : `while abs(a) >= eps`. Для знакозмінного ряду з монотонно спадними за модулем членами (ознака Лейбніца) похибка не перевищує модуля першого відкинутого члена. Для безпеки додають обмеження на кількість ітерацій — на випадок розбіжного ряду.

4. Формули для перевірки: $\sum_{k=1}^n k = n(n+1)/2$; $\sum_{k=1}^n k^2 = n(n+1)(2n+1)/6$; $\sum_{k=0}^n q^k = (q^{n+1} - 1)/(q - 1)$ при $q \neq 1$; нескінченна сума $\sum 1/k^2 = \pi^2/6$ (збігається повільно: похибка після n членів $\approx 1/n$); гармонічний ряд $\sum 1/k$ розбігається ($H_n \approx \ln n + 0,5772$).

5. Точність підсумовування. Через обмеженість точності float результат залежить від порядку додавання: додавання малого числа до великої суми втрачає молодші розряди. Тому члени, що спадають, точніше підсумовувати **від найменших до найбільших**. Функція

math.fsum виконує підсумовування з компенсацією похибки та дає максимально точний результат.

6. Добуток як набір множників. Біноміальний коефіцієнт $C(n, k) = \prod_{i=1}^k (n - k + i)/i$ обчислюють за ітераційною схемою $c = c * (n - k + i) // i$; на кожному кроці c – цілий біноміальний коефіцієнт $C(n - k + i, i)$, тому ділення націло точне.

Хід роботи

Завдання 1. Ручне обчислення сум і добутків

Обчислити вручну скінченні суми та добутки, виконавши трасування циклів накопичення.

Порядок виконання:

1. Обчисліть $S_1 = \sum_{k=1}^5 k^2$; складіть таблицю « k — k^2 — S » (S — часткова сума після додавання члена) і перевірте результат за формулою $n(n+1)(2n+1)/6$.
2. Обчисліть $S_2 = \sum_{k=0}^4 1/2^k$; складіть таблицю членів, визначте відповідь і порівняйте з формулою суми геометричної прогресії при $q = 1/2$ (чому S_2 менша за 2?).
3. Обчисліть добуток $P = \prod_{k=1}^5 (k+1)/k$; запишіть проміжні значення P і поясніть, чому більшість множників скорочується (телескопічний добуток); знайдіть загальну формулу для n множників.
4. Обчисліть $C(7, 3)$ як добуток за ітераційною схемою $c = c * (n - k + i) // i$ для $i = 1, 2, 3$; запишіть значення c після кожного кроку.
5. Для яких початкових значень накопичувачів результат порожньої суми й порожнього добутку буде коректним? Поясніть, чому для добутку початкове значення дорівнює 1.

Завдання 2. Трасування обчислення e^x із заданою точністю

Виконати вручну алгоритм обчислення e^1 за рядом Тейлора з точністю $\varepsilon = 0,01$.

```
алг Експонента
  ввести x, eps
  s := 0; a := 1; k := 0
  поки |a| >= eps
    s := s + a
    k := k + 1
    a := a * x / k
  все
  вивести s, k
кін
```

Порядок виконання:

1. Побудуйте таблицю трасування зі стовпцями « k », « a (поточний член)», « $|a| \geq \varepsilon?$ », « s ».
2. Виконайте алгоритм для $x = 1$, $\varepsilon = 0,01$ (члени обчислюйте з точністю до п'яти знаків після коми) до моменту, коли умова стане хибною.
3. Запишіть, скільки членів було додано та яке значення s отримано; порівняйте з точним значенням $e = 2,71828\dots$ і обчисліть фактичну абсолютну похибку; перевірте, що вона менша за ε .
4. Виконайте алгоритм для $x = 2$ з $\varepsilon = 0,1$ і порівняйте кількість членів із попереднім випадком.

5. Поясніть, чому перший відкинутий член ($a \approx 0,0083$) оцінює величину похибки, і чому така оцінка правильна для цього ряду (усі члени додатні й швидко спадають).

Завдання 3. Програмна реалізація алгоритмів сум і добутоків

Реалізувати мовою Python накопичення сум і добутоків, обчислення e^x та $\sin x$ за рядами з заданою точністю та біноміальні коефіцієнти.

Порядок виконання:

1. Напишіть функцію `sum_squares(n)` (цикл із накопичувачем суми) та перевірте її за формулою для n від 0 до 100.
2. Напишіть функцію `exp_series(x, eps)`, що обчислює e^x за рекурентною формулою та повертає пару (значення, кількість доданих членів); застосуйте обмеження кількості ітерацій на випадок помилки.
3. Напишіть функцію `sin_series(x, eps)` за рекурентною формулою для синуса; перевірте результат функцією `math.sin` і виведіть абсолютну похибку.
4. Напишіть функцію `binomial(n, k)` за схемою добутку й перевірте її за `math.comb` для кількох значень, а також для $C(52, 5)$ — кількості різних наборів із 5 карт у колоді.
5. Перевірте функції на тестах, у тому числі для граничних значень ($n = 0$, $x = 0$, $k = 0$).

Приклад реалізації мовою Python:

python

```
import math

def sum_squares(n):
    s = 0
    for k in range(1, n + 1):
        s += k * k
    return s

def exp_series(x, eps, limit=1000):
    s, a, k = 0.0, 1.0, 0
    while abs(a) >= eps and k < limit:
        s += a
        k += 1
        a = a * x / k
    return s, k

def sin_series(x, eps, limit=1000):
    s, a, k = 0.0, x, 0
    while abs(a) >= eps and k < limit:
        s += a
        k += 1
        a = -a * x * x / ((2 * k) * (2 * k + 1))
    return s, k

def binomial(n, k):
    c = 1
    for i in range(1, k + 1):
        c = c * (n - k + i) // i
    return c

ok = all(sum_squares(n) == n * (n + 1) * (2 * n + 1) // 6 for n in range(101))
```

```

print("sum_squares: збір з формулою для n = 0..100:", ok, "; sum_squares(5) =",
sum_squares(5))

e1, k1 = exp_series(1, 0.01)
print(f"exp_series(1, 0.01): {e1:.5f}, членів: {k1}, похибка {abs(e1 - math.e):.5f}")
e2, k2 = exp_series(2, 0.1)
print(f"exp_series(2, 0.1): {e2:.5f}, членів: {k2}, похибка {abs(e2 -
math.exp(2)):.5f}")
e0, k0 = exp_series(0, 1e-9)
print(f"exp_series(0, 1e-9): {e0}, членів: {k0}")

s, k = sin_series(1, 1e-8)
print(f"sin_series(1, 1e-8): {s:.10f}, членів: {k}, похибка {abs(s -
math.sin(1)):.2e}")
s, k = sin_series(math.pi / 6, 1e-8)
print(f"sin_series(pi/6, 1e-8): {s:.10f}, членів: {k}")

print("binomial(7, 3) =", binomial(7, 3), "; binomial(10, 0) =", binomial(10, 0))
print("binomial(52, 5) =", binomial(52, 5), "; збір з math.comb:", binomial(52, 5) ==
math.comb(52, 5))

```

Результат виконання програми:

```

sum_squares: збір з формулою для n = 0..100: True ; sum_squares(5) = 55
exp_series(1, 0.01): 2.70833, членів: 5, похибка 0.00995
exp_series(2, 0.1): 7.26667, членів: 6, похибка 0.12239
exp_series(0, 1e-9): 1.0, членів: 1
sin_series(1, 1e-8): 0.8414709846, членів: 6, похибка 1.60e-10
sin_series(pi/6, 1e-8): 0.4999999919, членів: 4
binomial(7, 3) = 35 ; binomial(10, 0) = 1
binomial(52, 5) = 2598960 ; збір з math.comb: True

```

Висновок за результатами: перевірка за формулою підтвердила коректність накопичення суми: $\text{sum_squares}(5) = 55$. Ряд для e^1 з точністю 0,01 потребує 5 членів (результат 2.70833 збігається з ручним трасуванням, фактична похибка 0.00995 менша за $\varepsilon = 0,01$), а для $x = 0$ додається лише перший член, тому результат дорівнює 1.0 за один крок. Для синуса достатньо кількох членів ряду, щоб досягти похибки порядку 10^{-9} або меншої. Рекурентні формули замінюють обчислення степенів і факторіалів на два множення й одне ділення на крок, а біноміальний коефіцієнт $C(52, 5) = 2\,598\,960$ обчислюється точно без виходу за межі цілих чисел завдяки цілочисельному діленню на кожному кроці.

Завдання 4. Дослідження точності та порядку підсумовування

Дослідити залежність кількості членів ряду від заданої точності, швидкість збіжності ряду $\sum 1/k^2$ та вплив порядку підсумовування на точність результату.

Порядок виконання:

1. Для $\varepsilon = 10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}$ обчисліть e^1 і $\sin 1$ за рядами та занесіть у таблицю кількість членів і абсолютну похибку відносно math.e та $\text{math.sin}(1)$.
2. Обчисліть часткові суми $S_N = \sum_{k=1}^N 1/k^2$ для $N = 10, 100, 1000, 10^6$ і порівняйте з $\pi^2/6$; перевірте, що помилка приблизно дорівнює $1/N$.
3. Підсумуйте $1/k^2$ для k від 1 до 10^6 у прямому порядку (від 1 до N) та у зворотному (від N до 1), порівняйте обидва результати з math.fsum і визначте, який порядок дає меншу похибку.
4. Обчисліть часткові суми гармонічного ряду H_n для $n = 10, 1000, 10^6$ та порівняйте з $\ln n + 0,5772$; зробіть висновок про розбіжність цього ряду.
5. Сформулюйте рекомендації щодо вибору порядку підсумовування та критерію зупинки.

Приклад реалізації мовою Python:

python

```
import math

print("  eps      членів(e)  похибка(e)   членів(sin 1)  похибка(sin 1)")
for p in (2, 4, 6, 8, 10):
    eps = 10.0 ** -p
    e, ke = exp_series(1, eps)
    s, ks = sin_series(1, eps)
    print(f"1e-{p:02d}   {ke:>9}   {abs(e - math.e):>11.2e}   {ks:>12}   {abs(s - math.sin(1)):>13.2e}")

print("\n      N      S_N      pi^2/6 - S_N      1/N")
target = math.pi ** 2 / 6
for n in (10, 100, 1000, 10 ** 6):
    sn = math.fsum(1 / k ** 2 for k in range(1, n + 1))
    print(f"{n:>8}   {sn:.10f}   {target - sn:.3e}   {1 / n:.3e}")

N = 10 ** 6
forward = 0.0
for k in range(1, N + 1):
    forward += 1 / k ** 2
backward = 0.0
for k in range(N, 0, -1):
    backward += 1 / k ** 2
exact = math.fsum(1 / k ** 2 for k in range(1, N + 1))
print("\nПрямий порядок: похибка =", abs(forward - exact))
print("Зворотний порядок: похибка =", abs(backward - exact))

print("\n      n      H_n      ln n + 0.5772")
for n in (10, 1000, 10 ** 6):
    h = math.fsum(1 / k for k in range(1, n + 1))
    print(f"{n:>7}   {h:>9.5f}   {math.log(n) + 0.5772:>12.5f}")
```

Результат виконання програми:

eps	членів(e)	похибка(e)	членів(sin 1)	похибка(sin 1)
1e-02	5	9.95e-03	2	8.14e-03
1e-04	8	2.79e-05	4	2.73e-06
1e-06	10	3.03e-07	5	2.49e-08
1e-08	12	2.26e-09	6	1.60e-10
1e-10	14	1.23e-11	7	7.62e-13

N	S_N	pi^2/6 - S_N	1/N
10	1.5497677312	9.517e-02	1.000e-01
100	1.6349839002	9.950e-03	1.000e-02
1000	1.6439345667	9.995e-04	1.000e-03
1000000	1.6449330668	1.000e-06	1.000e-06

Прямий порядок: похибка = 4.3520742565306136e-14
Зворотний порядок: похибка = 2.220446049250313e-16

n	H_n	ln n + 0.5772
10	2.92897	2.87979
1000	7.48547	7.48496
1000000	14.39273	14.39271

Висновок за результатами: при зменшенні ϵ у 100 разів кількість членів ряду зростає лише на 1–3, бо члени ряду Тейлора спадають швидше за геометричну прогресію: для $\epsilon = 10^{-8}$ потрібно 12 членів ряду для e^1 і 6 – для $\sin 1$. Ряд $\sum 1/k^2$ збігається значно повільніше – похибка S_N приблизно дорівнює $1/N$ (стовпці « $\pi^2/6 - S_N$ » та « $1/N$ » майже збігаються), тому для однієї додаткової цифри точності потрібно в 10 разів більше членів. Сума у зворотному порядку (від малих членів до великих) має

меншу похибку ($2.2e-16$ проти $4.4e-14$ у прямому порядку), а `math.fsum` виконує підсумовування з компенсацією. Гармонічний ряд не збігається: H_n зростає як $\ln n + 0,5772$ без обмеження.

Контрольні питання

1. З яких значень починають накопичувачі суми та добутку? Яким є результат порожньої суми та порожнього добутку?
2. Що таке рекурентне співвідношення для членів ряду? Чому воно ефективніше за обчислення кожного члена «з нуля»?
3. Запишіть рекурентні формули для членів ряду e^x та $\sin x$.
4. Як обчислюється сума ряду із заданою точністю ϵ ? Що є критерієм зупинки?
5. Чому для знакозмінного ряду Лейбніца перший відкинутий член оцінює похибку?
6. Яка швидкість збіжності ряду $\sum 1/k^2$? Чи збігається гармонічний ряд?
7. Чому порядок підсумовування впливає на результат при використанні чисел з плаваючою комою? Яка функція Python виконує точне підсумовування?
8. Як обчислюють біноміальний коефіцієнт як добуток? Чому ділення на кожному кроці точне?

Зміст звіту

1. Тема, мета роботи.
2. Таблиці трасування сум і ряду e^x (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Таблиця залежності кількості членів ряду та похибки від ϵ .
5. Висновки.